

Problem Definition

Yulu is India’s leading micro-mobility service provider, offering shared electric cycles for sustainable commuting. Recently, Yulu has observed a dip in revenues. They want to identify the **factors that significantly influence the demand (count)** for electric cycles in the Indian market.

Objective:

- Find which variables affect cycle rentals (count) — e.g., workingday , season , weather , temp , humidity , etc.
- Quantify how well these variables explain demand.
- Provide business recommendations for fleet allocation and pricing.

Approach:

- Perform Exploratory Data Analysis (EDA).
- Conduct hypothesis testing (t-test, ANOVA, Chi-square).
- Interpret results and translate into actionable insights.

Import Libraries & Load Dataset

```
In [20]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from scipy.stats import ttest_ind, f_oneway, chi2_contingency, shapiro, levene, kruskal
from statsmodels.stats.multicomp import pairwise_tukeyhsd

# Load dataset
df = pd.read_csv("yulu_data.csv")

# Preview
df.head()
```

Out[20]:

	datetime	season	holiday	workingday	weather	temp	atemp	humidity	windspeed	casual	registered	count
0	2011-01-01 00:00:00	1	0	0	1	9.84	14.395	81	0.0	3	13	16
1	2011-01-01 01:00:00	1	0	0	1	9.02	13.635	80	0.0	8	32	40
2	2011-01-01 02:00:00	1	0	0	1	9.02	13.635	80	0.0	5	27	32
3	2011-01-01 03:00:00	1	0	0	1	9.84	14.395	75	0.0	3	10	13
4	2011-01-01 04:00:00	1	0	0	1	9.84	14.395	75	0.0	0	1	1

EDA — Data Overview

We check shape, column types, missing values, and statistical summary.

```
In [5]: print("Shape:", df.shape)
df.info()
df.isnull().sum()
df.describe()

# **Observations:**
# - Dataset has 10886 rows and Y 12.
# - Categorical variables: `season`, `holiday`, `workingday`, `weather`.
# - No/Some missing values.
# - Continuous variables (`temp`, `atemp`, `humidity`, `windspeed`, `count`) have ranges ... (comment on min, max, outliers).
```

Shape: (10886, 12)
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 10886 entries, 0 to 10885
Data columns (total 12 columns):
Column Non-Null Count Dtype
--- -
0 datetime 10886 non-null object
1 season 10886 non-null category
2 holiday 10886 non-null category
3 workingday 10886 non-null category
4 weather 10886 non-null category
5 temp 10886 non-null float64
6 atemp 10886 non-null float64
7 humidity 10886 non-null int64
8 windspeed 10886 non-null float64
9 casual 10886 non-null int64
10 registered 10886 non-null int64
11 count 10886 non-null int64
dtypes: category(4), float64(3), int64(4), object(1)
memory usage: 723.7+ KB

Out[5]:

	temp	atemp	humidity	windspeed	casual	registered	count
count	10886.00000	10886.000000	10886.000000	10886.000000	10886.000000	10886.000000	10886.000000
mean	20.23086	23.655084	61.886460	12.799395	36.021955	155.552177	191.574132
std	7.79159	8.474601	19.245033	8.164537	49.960477	151.039033	181.144454
min	0.82000	0.760000	0.000000	0.000000	0.000000	0.000000	1.000000
25%	13.94000	16.665000	47.000000	7.001500	4.000000	36.000000	42.000000
50%	20.50000	24.240000	62.000000	12.998000	17.000000	118.000000	145.000000
75%	26.24000	31.060000	77.000000	16.997900	49.000000	222.000000	284.000000
max	41.00000	45.455000	100.000000	56.996900	367.000000	886.000000	977.000000

Data Preprocessing

Convert categorical columns to category type.

```
In [3]: categorical_cols = ['season', 'holiday', 'workingday', 'weather']
for col in categorical_cols:
    df[col] = df[col].astype('category')
```

Univariate Analysis

1. Distribution of Continous variables.

```
In [29]: cont_cols = ['temp', 'atemp', 'humidity', 'windspeed', 'count']

for col in cont_cols:
    plt.figure(figsize=(6,4))
    sns.histplot(df[col], kde=True)
    plt.title(f"Distribution of {col}")
    plt.show()

### Observations on Continuous Variables

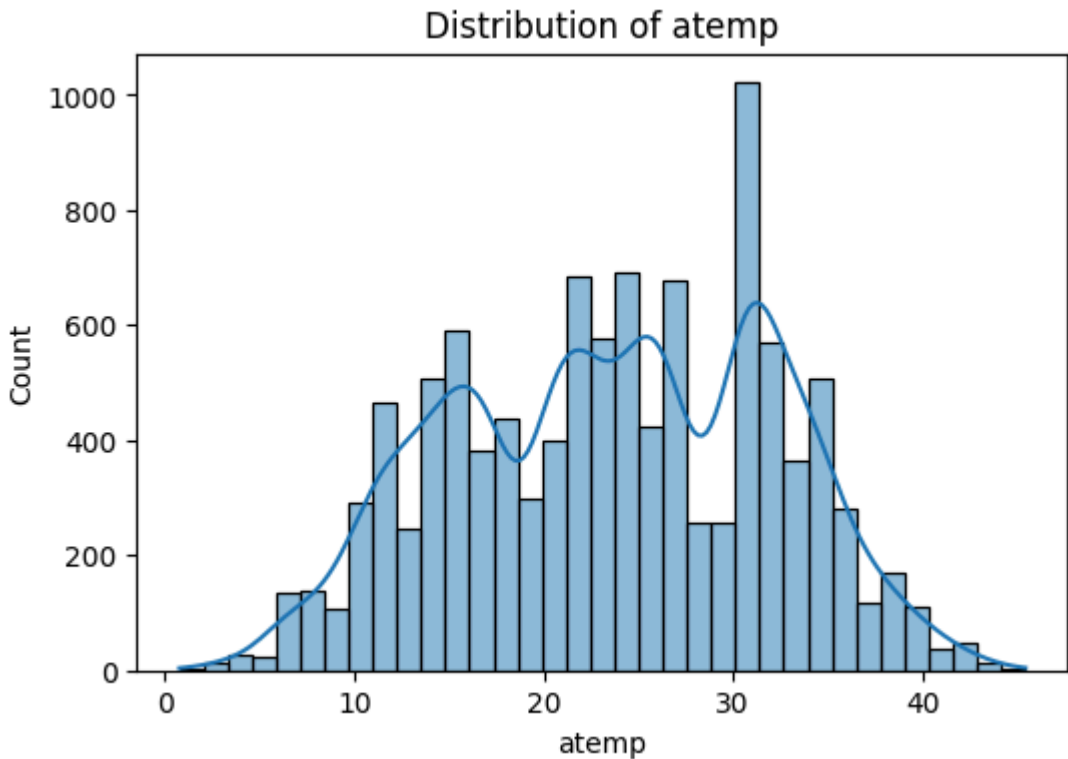
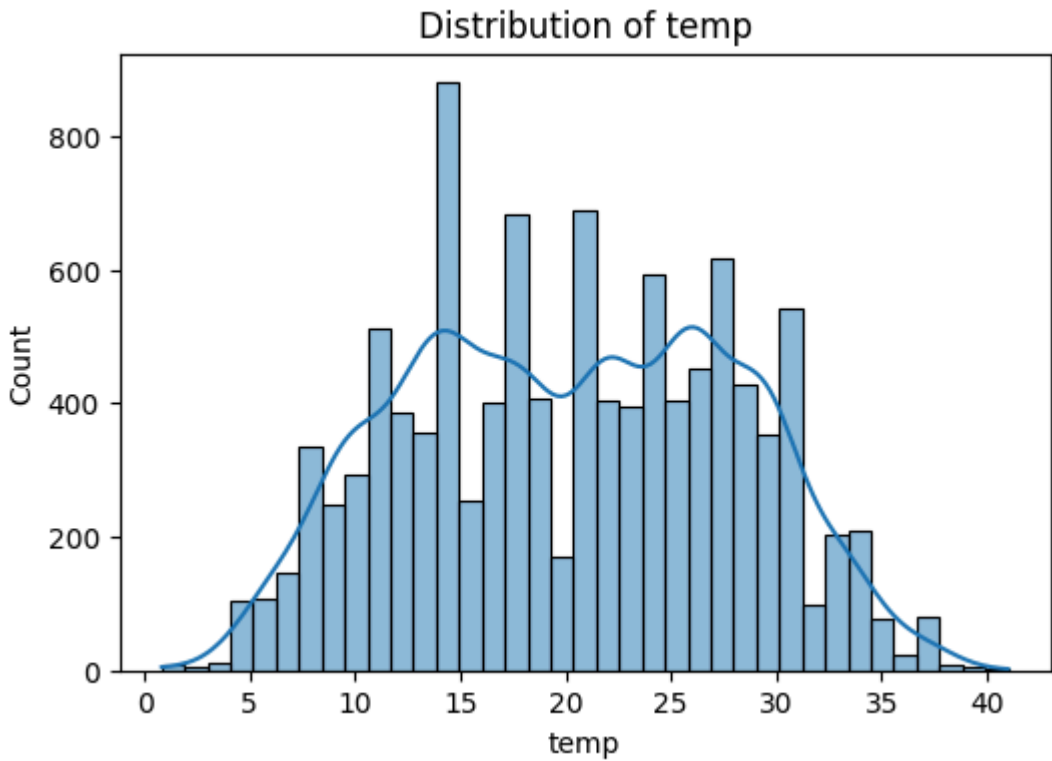
# 1. **Temperature (`temp`)**
# - Distribution is close to normal with values ranging from ~0°C to ~40°C.
# - Peak demand occurs around **15-30°C**, which is comfortable for commuting.
# - Very few observations at extreme cold (<5°C) or very hot (>35°C).

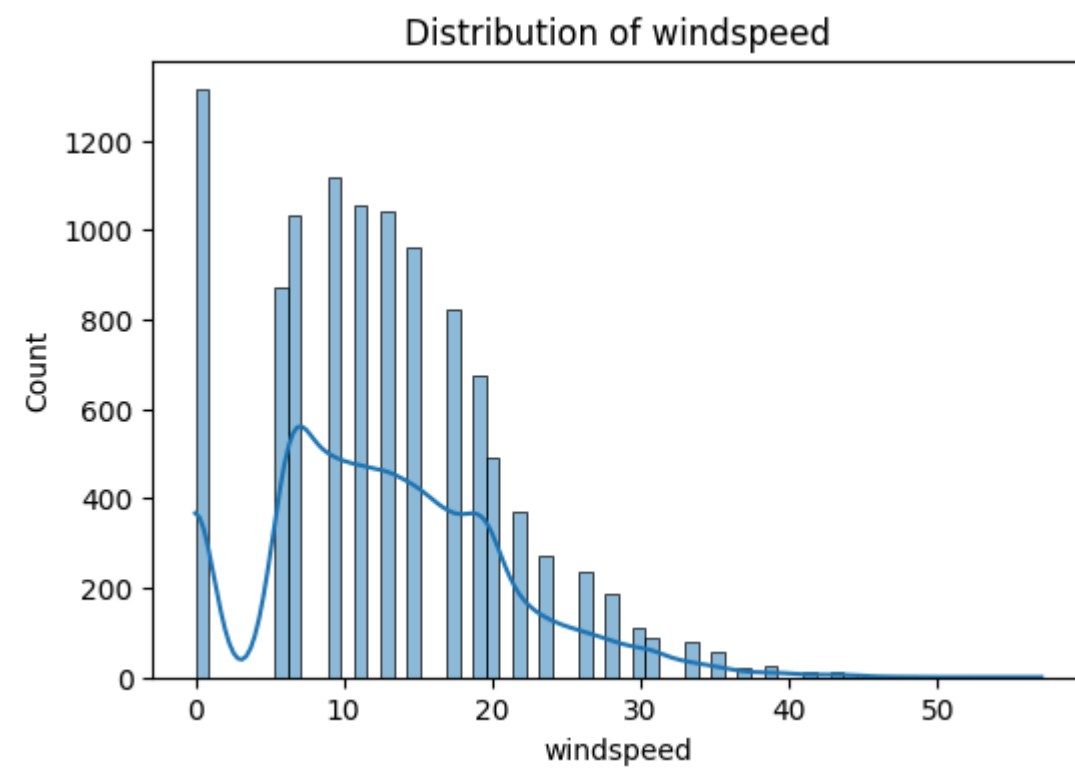
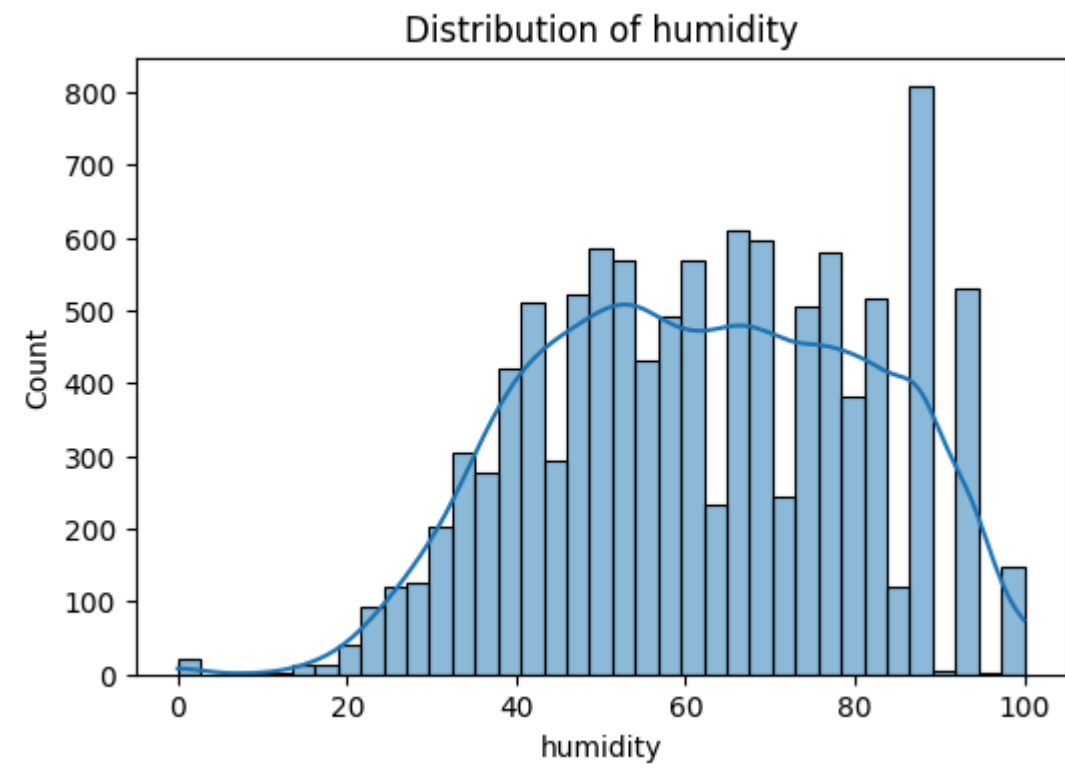
# 2. **Feeling Temperature (`atemp`)**
# - Distribution is very similar to `temp` (as expected).
# - Ranges from ~0 to ~45°C.
# - Slightly more concentrated between **20-30°C**, suggesting most rides happen in moderate perceived temperature.

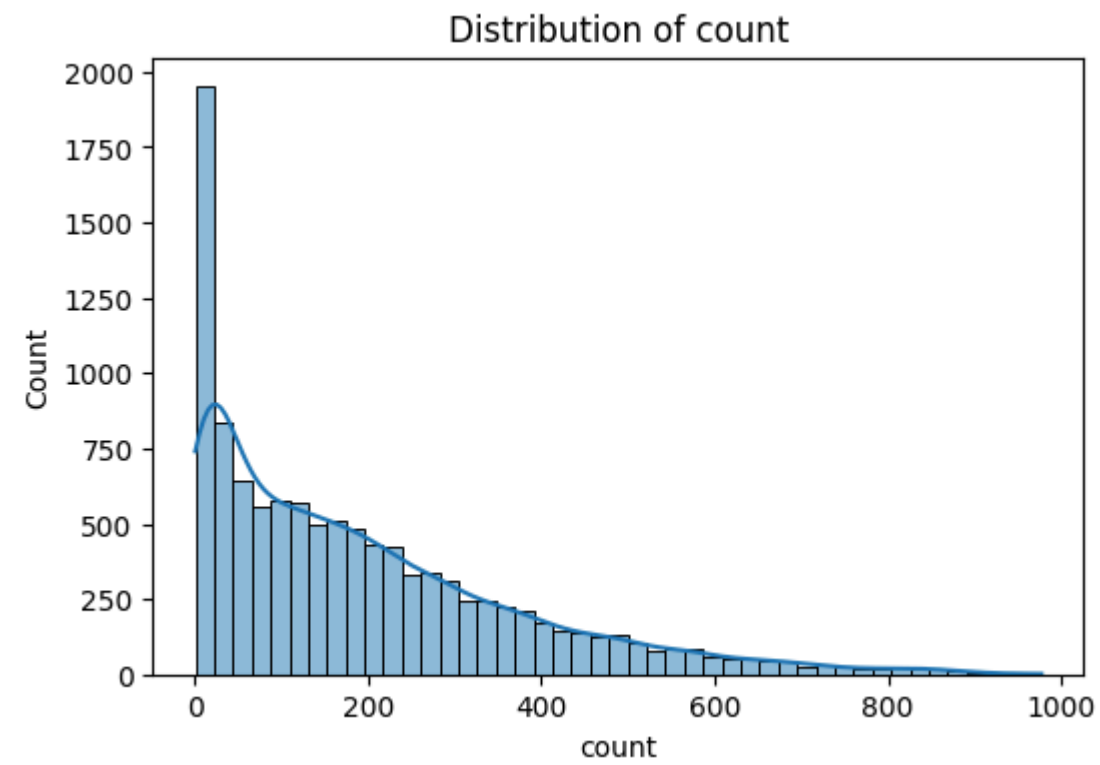
# 3. **Humidity**
# - Values spread across 0-100%.
# - Most rentals occur between **40-80% humidity**.
# - Very high humidity (>90%) is less frequent → possible discomfort for cycling.

# 4. **Windspeed**
# - Distribution is **right-skewed**.
# - Most days have **low windspeed (0-20)**.
# - Very few observations with windspeed >30 → indicates riders avoid extreme windy conditions.

# 5. **Count (Total Rentals)**
# - Distribution is **highly right-skewed**.
# - Most values are concentrated below **500 rides per day**.
# - Some extreme values above 800-900 suggest occasional **peak demand days**.
# - This indicates strong variation in demand and possible presence of outliers.
```







2. Distribution of Categorical variables.

```
In [31]: for col in categorical_cols:
plt.figure(figsize=(6,4))
sns.countplot(x=col, data=df)
plt.title(f"Distribution of {col}")
plt.show()
```

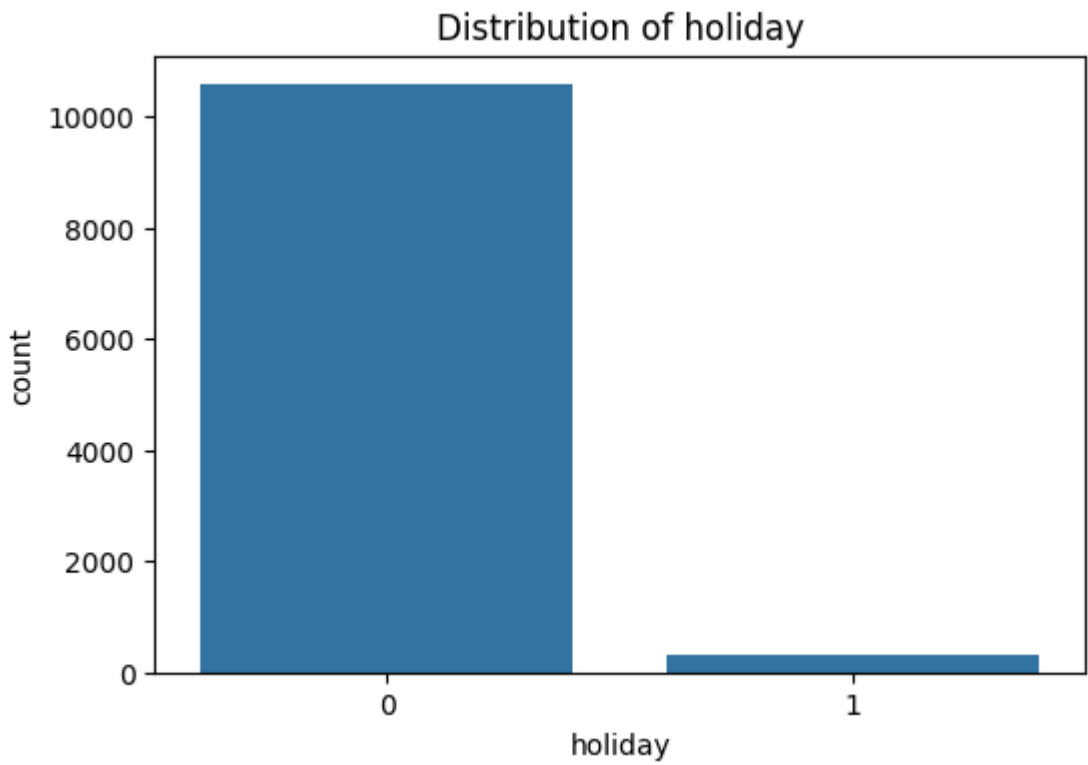
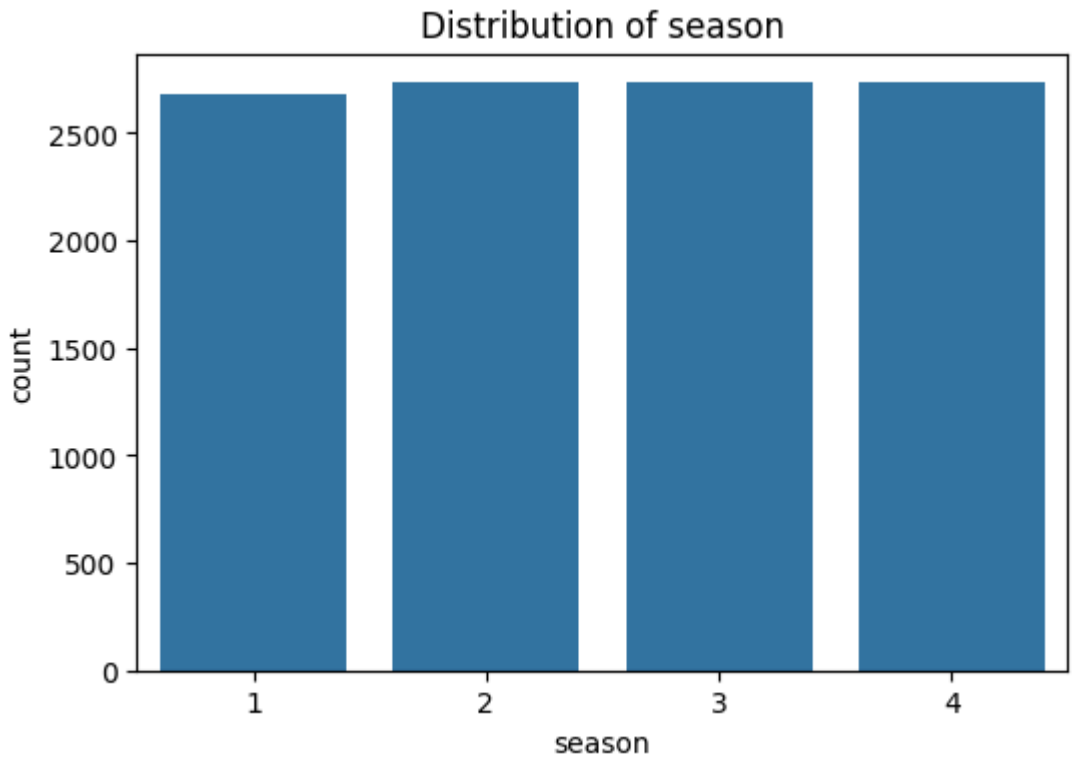
Observations on Categorical Variables

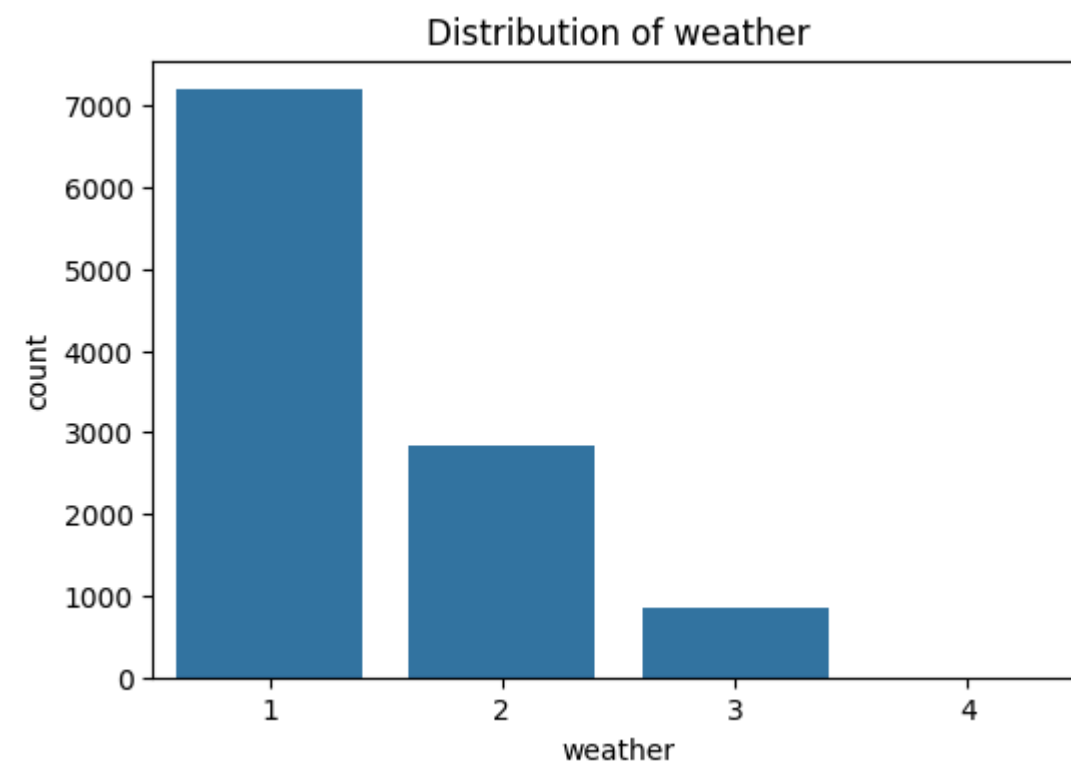
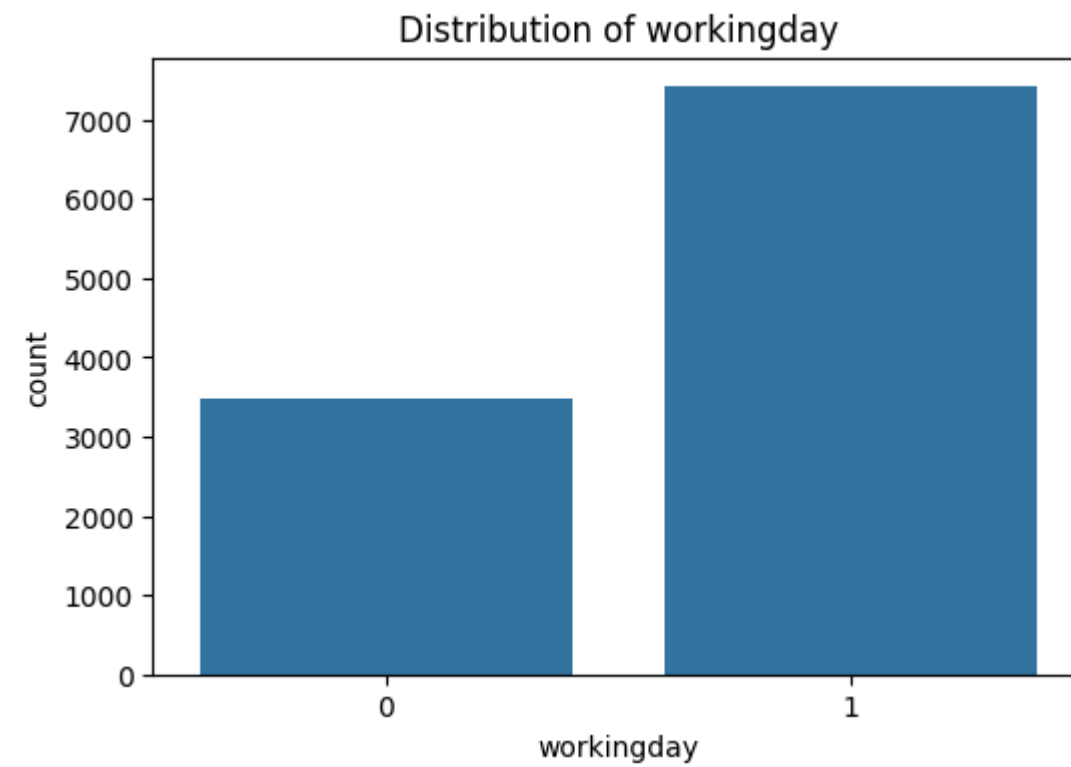
Season - Rentals are fairly balanced across all seasons; no major difference in data size.

Holiday - Very few holidays compared to non-holidays.

Workingday - More rides logged on working days than non-working days.

Weather - Most rides occur in clear/partly cloudy (1), fewer in misty (2), very few in rainy/snowy (3-4).





Bivariate Analysis

We explore relation of demand (`count`) with workingday, season, and weather.

```
In [30]: sns.boxplot(x='workingday', y='count', data=df)
plt.title("Workingday vs Count")
plt.show()

sns.boxplot(x='season', y='count', data=df)
plt.title("Season vs Count")
plt.show()
```

```
sns.boxplot(x='weather', y='count', data=df)
plt.title("Weather vs Count")
plt.show()
```

Observations

Workingday vs Count:

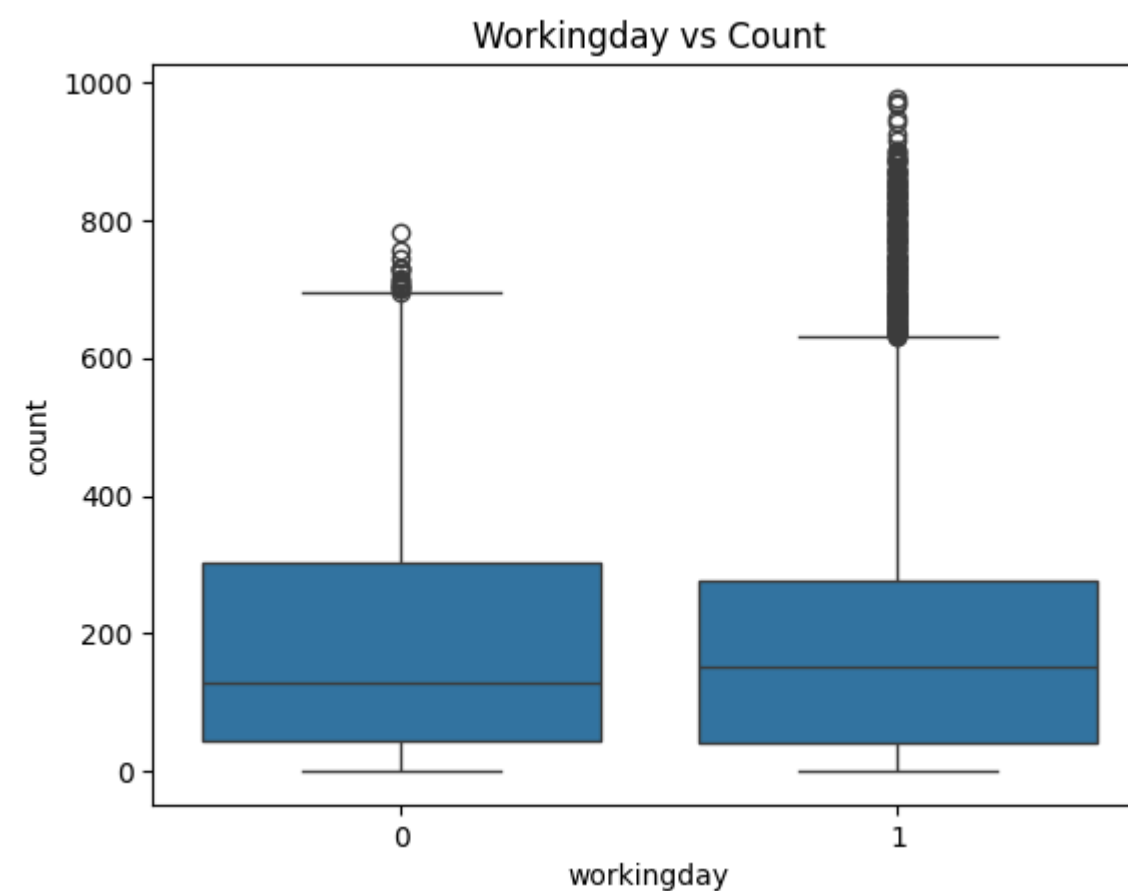
The median count of bikes rented is slightly higher on non-working days (0) compared to working days (1).
 # The distribution on working days shows more outliers with higher counts, indicating sporadic high demand.

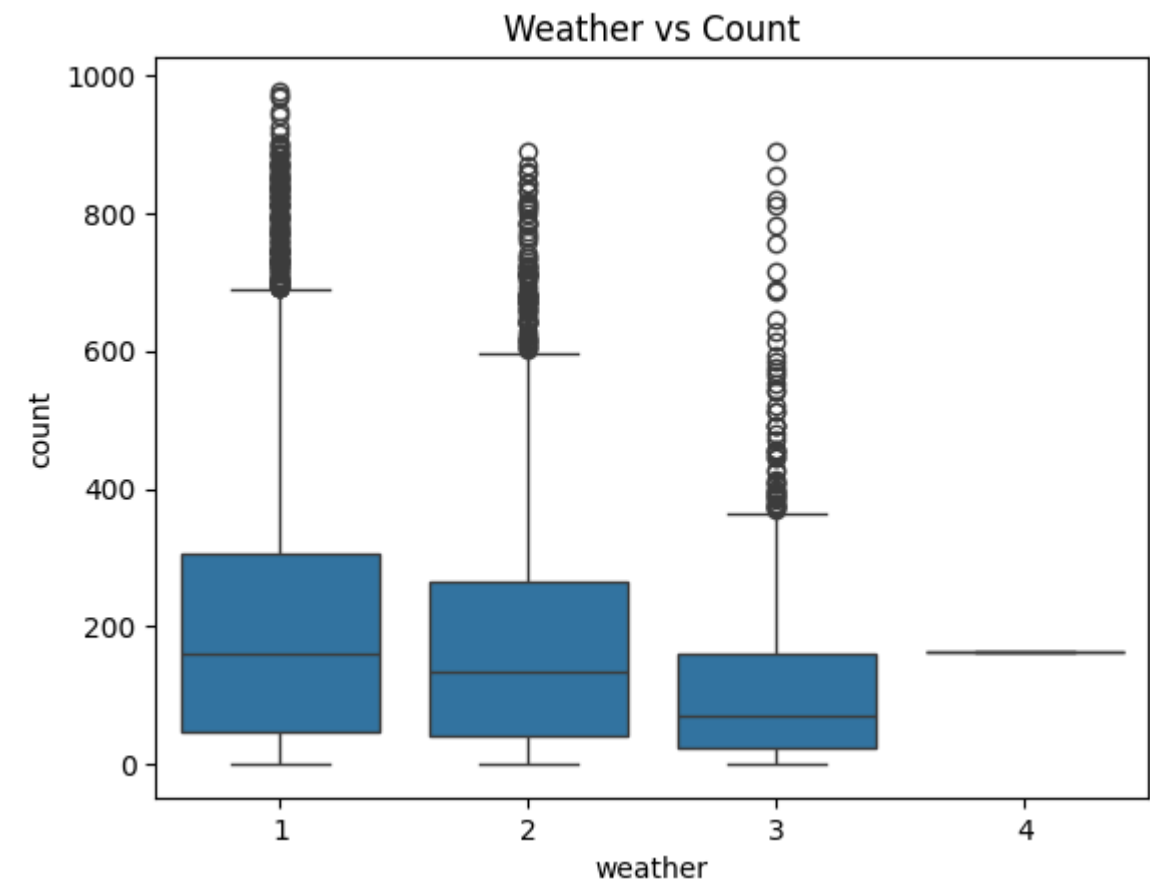
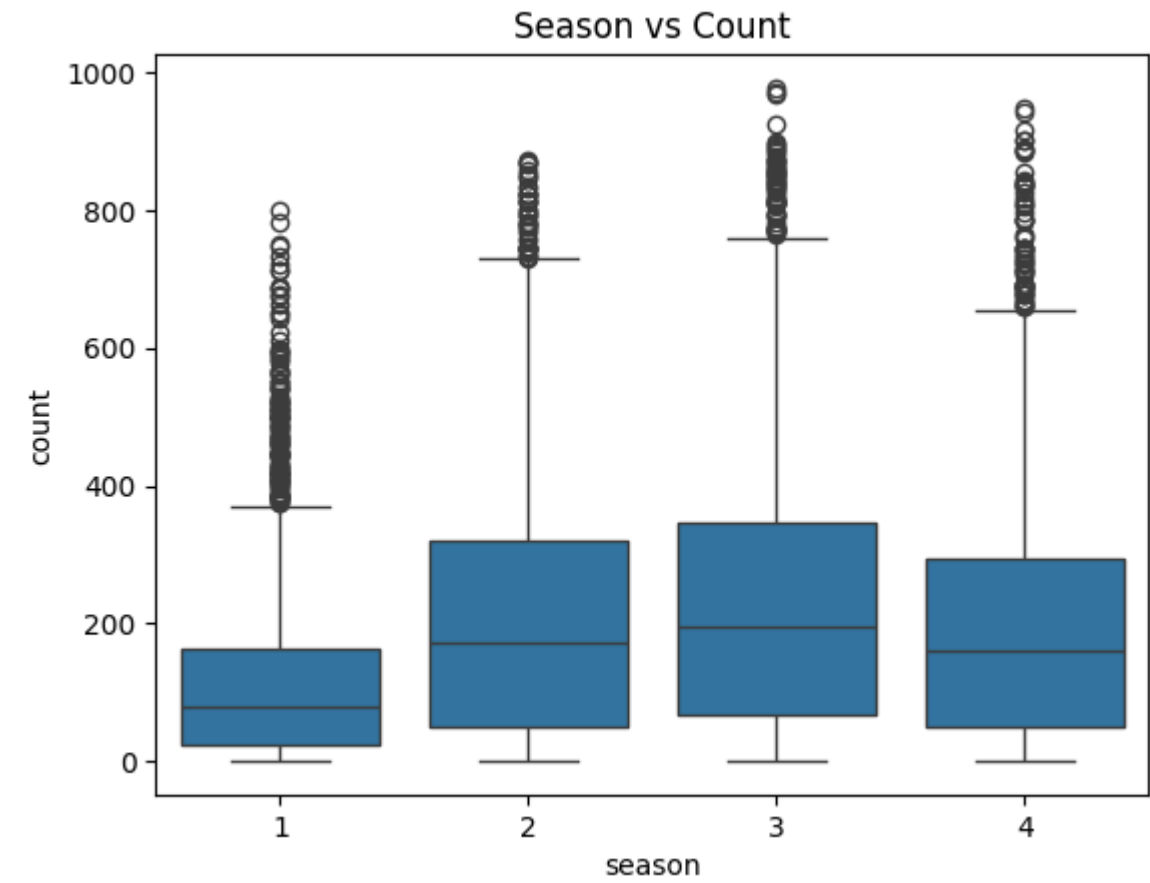
Season vs Count:

Seasons 2 and 3 have higher median bike rental counts compared to seasons 1 and 4.
 # Season 3 has the widest spread and many high count outliers, suggesting peak demand in that season.
 # Season 1 shows the lowest bike rental counts overall.

Weather vs Count:

Weather conditions 1 and 2 have higher median rental counts than weather 3.
 # Weather 4 shows no variability and very low or zero rentals, likely indicating extreme weather.
 # Rental counts decrease as weather conditions worsen (from 1 to 3).





Hypothesis Testing

1. 2- Sample T-Test

```
In [18]: from scipy import stats
# Split data into two groups
workingday_rentals = df[df['workingday'] == 1]['count']
non_workingday_rentals = df[df['workingday'] == 0]['count']

# Perform 2-sample T-test (Welch's test)
t_stat, p_val = stats.ttest_ind(workingday_rentals, non_workingday_rentals, equal_var=False)

print("T-statistic:", t_stat)
print("P-value:", p_val)

# Conclusion
alpha = 0.05
if p_val < alpha:
    print("Reject H0: Working day has a significant effect on rentals.")
else:
    print("Fail to reject H0: No significant effect of working day on rentals.")
```

T-statistic: 1.2362580418223226

P-value: 0.21640312280695098

Fail to reject H0: No significant effect of working day on rentals.

2. ANOVA

- Anova for weather

```
In [23]: # ---- 1. Normality check (Shapiro-Wilk) for weather groups ----
for cat in df['weather'].unique():
    stat, p = stats.shapiro(df.loc[df['weather'] == cat, 'count'])
    print(f"Weather {cat} - Shapiro p-value: {p}")

## observation
# Since all groups fail normality (p << 0.05) →
# We cannot use regular ANOVA.
# Instead, we should use a non-parametric test: Kruskal-Wallis
```

Weather 1 - Shapiro p-value: 1.5964921477006555e-57

Weather 2 - Shapiro p-value: 9.777839106111785e-43

Weather 3 - Shapiro p-value: 3.875893017396149e-33

Weather 4 - Shapiro p-value: nan

/tmp/ipython-input-3042731140.py:3: SmallSampleWarning: One or more sample arguments is too small; all returned values will be NaN. See documentation for sample size requirements.

```
stat, p = stats.shapiro(df.loc[df['weather'] == cat, 'count'])
```

```
In [25]: # --- Kruskal-Wallis test for Weather ----
from scipy.stats import kruskal
# H0 : No significant difference, i.e., rentals are similar across groups.
# Ha : There is a significant difference, i.e., rentals are not similar across groups.

# Group counts by weather categories
groups_weather = [df.loc[df['weather'] == cat, 'count'] for cat in df['weather'].unique()]

stat, p = kruskal(*groups_weather)
print("Statistic:", stat, "p-value:", p)

# Since your p-value = 3.5e-44 (which is way smaller than 0.05):
# Reject H0 → There is a significant difference, i.e., rentals are not similar across groups.
```

Statistic: 205.00216514479087 p-value: 3.501611300708679e-44

- **Anova for season**

```
In [26]: # ---- 1. Normality check (Shapiro-Wilk) for season groups ----
for cat in df['season'].unique():
    stat, p = stats.shapiro(df.loc[df['season'] == cat, 'count'])
    print(f"Season {cat} - Shapiro p-value: {p}")

## observation
# Since all groups fail normality (p << 0.05) →
# We cannot use regular ANOVA.
# Instead, we should use a non-parametric test: Kruskal-Wallis
```

Season 1 - Shapiro p-value: 8.749584618867662e-49
 Season 2 - Shapiro p-value: 6.039374406270491e-39
 Season 3 - Shapiro p-value: 1.043680518918597e-36
 Season 4 - Shapiro p-value: 1.1299244409282836e-39

```
In [27]: # --- Kruskal-Wallis test for Season ----
from scipy.stats import kruskal
# H0 : No significant difference, i.e., rentals are similar across groups.
# Ha : There is a significant difference, i.e., rentals are not similar across groups.

# Group counts by season categories
groups_season = [df.loc[df['season'] == cat, 'count'] for cat in df['season'].unique()]

# Run Kruskal-Wallis test
stat, p = kruskal(*groups_season)

print("Kruskal-Wallis test for Season")
print("Statistic:", stat, "p-value:", p)

## Observation
# Your p-value = 2.48e-151 is extremely small (<< 0.05).
# Conclusion: Reject H0 → There is a significant difference, i.e., rentals are not similar across groups.
```

Kruskal-Wallis test for Season
 Statistic: 699.6668548181988 p-value: 2.479008372608633e-151

3. Chi-square test

```
In [28]: import pandas as pd
from scipy.stats import chi2_contingency

# (H0): Season and Weather are independent.
# (H1): Season and Weather are dependent (association exists).

# Create contingency table (cross-tabulation of Season vs Weather)
contingency_table = pd.crosstab(df['season'], df['weather'])

print("Contingency Table:")
print(contingency_table)

# Run Chi-square test
chi2, p, dof, expected = chi2_contingency(contingency_table)

print("\nChi-square Test for Season vs Weather")
print("Chi2 Statistic:", chi2)
print("Degrees of Freedom:", dof)
```

```
print("p-value:", p)
print("\nExpected Frequencies:")
print(expected)

## observation
# p-value = 1.55e-07 < 0.05 → reject H0.
# Conclusion: Weather is dependent on Season
```

Contingency Table:

weather	1	2	3	4
season				
1	1759	715	211	1
2	1801	708	224	0
3	1930	604	199	0
4	1702	807	225	0

Chi-square Test for Season vs Weather
Chi2 Statistic: 49.158655596893624
Degrees of Freedom: 9
p-value: 1.549925073686492e-07

Expected Frequencies:
[[1.77454639e+03 6.99258130e+02 2.11948742e+02 2.46738931e-01]
[1.80559765e+03 7.11493845e+02 2.15657450e+02 2.51056403e-01]
[1.80559765e+03 7.11493845e+02 2.15657450e+02 2.51056403e-01]
[1.80625831e+03 7.11754180e+02 2.15736359e+02 2.51148264e-01]]

Insights & Conclusion

Temperature & Comfort Range

- Rentals peak between 15–30°C, which is a comfortable commuting range.
- Insight: Yulu can expect maximum demand during moderate temperatures and should adjust fleet availability accordingly. Extreme weather (too hot or too cold) reduces rentals.

Humidity & Windspeed

- Most rides occur when humidity is 40–80% and windspeed is low (<20).
- Insight: Unfavorable conditions (very humid or very windy days) discourage cycling. Yulu may consider offering discounts/promotions on such days to sustain demand.

Working Days vs Non-working Days

- Median rentals are slightly higher on non-working days, but working days show
- higher variability with occasional demand spikes.
- Insight: On weekends/holidays, Yulu should focus more on leisure areas (parks, malls). On working days, it should allocate more bikes near offices and metro stations to meet commuter demand.

Seasonal Effect

- showing many peak outliers.
- Demand is highest in Season 3 (Fall) and Season 2 (Summer), with Season 3
- Demand is lowest in Season 1 (Spring).
- Insight: Yulu should plan fleet expansion and maintenance cycles around seasonal trends (e.g., add more vehicles in Summer/Fall, schedule servicing during Spring).

Weather Impact

- Weather category 4 (extreme) results in nearly zero rentals.
- weather worsens (misty → rainy/snowy → extreme).
- Rentals are highest in clear/partly cloudy weather and decrease steadily as
- Insight: Weather is a strong predictor of demand. Yulu should use real-time weather forecasts to predict demand and adjust fleet distribution dynamically.

Weather–Season Relationship

- The Chi-square test shows that weather is dependent on season.
- Insight: Since weather patterns vary by season, combining both season + weather will improve demand forecasting models.

🔑 Final Business Recommendation

- Fleet Planning: Increase bikes in high-demand seasons (summer/fall), reduce in spring/winter.
- Dynamic Demand Forecasting: Incorporate temperature, weather, humidity, and season into prediction models.
- Targeted Distribution: Allocate bikes near offices on weekdays and recreational spots on weekends.
- Promotions/Offer: Run discounts on days with bad weather (high humidity, heavy rain) to stabilize usage.
- List itemOperational Strategy: Schedule bike maintenance during low-demand periods (e.g., extreme weather days, spring season).